

Chương 3 (tt)

Function

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

Email: thept@hcmute.edu.vn

NỘI DUNG

- Khái niệm về Hàm
- Các loại hàm
- Các loại giá trị trả về của UFDs
- Tạo và quản lý hàm UFDs
- Scalar Function
- Table-valued Function
- Sử dụng hàm UFDs

Khái niệm về Hàm

- Hàm tương tự thủ tục bao gồm các phát biểu T-SQL và một số cấu trúc điều khiển được lưu với một tên và được xử lý như một đơn vị độc lập. Hàm được biên dịch trước, không cần kiểm tra và biên dịch lại.
- Điểm khác biệt giữa hàm và thủ tục là hàm trả về một giá trị thông qua tên hàm còn thủ tục thì không.

Khái niệm về Hàm

- **Hàm được dùng trong:**
 - Lệnh print hay lệnh Select để hiển thị giá trị trả về của hàm.
 - Danh sách chọn của một câu lệnh Select để cho ra một giá trị.
 - Một điều kiện tìm kiếm của mệnh đề Where trong các câu lệnh T-SQL.

Ưu điểm của Hàm

- Người gọi chỉ gọi một câu lệnh đơn và SQL Server chỉ kiểm tra một lần sau đó tạo ra một execute plan và thực thi. Cú pháp của các câu lệnh SQL đã được SQL Sever kiểm tra trước khi save nên nó không cần kiểm lại khi thực thi → giảm nghẽn mạng
- Bảo trì (maintainability) dễ dàng hơn do việc tách rời giữa business rules và database. Nếu có một sự thay đổi nào đó về mặt logic thì ta chỉ việc thay đổi code bên trong hàm
- Security: có thể được encrypt (mã hóa) để tăng cường tính bảo mật.

Các loại Hàm

- Có hai loại:
 - **Built-in functions:** Hoạt động như là một định nghĩa trong T-SQL và không thể hiệu chỉnh. Chỉ được tham chiếu trong các câu lệnh T-SQL. Trị trả về là một tập các dòng(Rowset), vô hướng(scalar) và aggregate(thống kê).
 - **User-define functions** hay còn gọi là **UDFs:** do người dùng tự định nghĩa để đáp ứng một mục tiêu nào đó. Các tham số truyền vào không được mang thuộc tính OUTPUT, do đó giá trị trả về cho hàm bằng phát biểu RETURN. Giá trị trả về là giá trị vô hướng (Scalar valued) hay bảng (Table – valued).

Các loại giá trị trả về của UFDs

- **Scalar Function:** Một hàm vô hướng trả về một giá trị đơn và có thể được dùng bất cứ nơi nào của biểu thức hay có thể được dùng câu lệnh SELECT, mệnh đề SET của lệnh UPDATE,... Một hàm vô hướng có thể được xem như kết quả của vài phép toán hay hàm chuỗi.
- **Table-valued Function :** Một hàm có giá trị trả về là một tập kết quả và có thể được dùng bất cứ nơi nào mà bảng hay view được dùng. Hàm giá trị bảng có thể được tham chiếu trong mệnh đề FROM của câu lệnh SELECT.
- Tên và những thông tin về Function khi được tạo ra sẽ chứa trong SysObjects table còn phần text của nó chứa trong SysComments table.

Các lệnh tạo và quản lý UDF

- Tạo Hàm
 - **CREATE FUNCTION <TenHam>...**
- Sửa Hàm
 - **ALTER FUNCTION <TenHam>...**
- Xóa Hàm
 - **DROP FUNCTION statement**
- Thực thi Hàm
 - Dùng lệnh **Print**
 - Dùng Lệnh **Select**
- Xem các lệnh của UDFs
 - **Sp-helptext TenHam**

Scalar Function

- Scalar Function (Không có tham số)
 - Là hàm không nhận giá trị từ bên ngoài truyền vào.
 - Cú pháp:

```
CREATE FUNCTION [Owner_name.]function_name  
RETURNS scalar_return_data_type  
[WITH { ENCRYPTION | SCHEMABINDING } ]  
[ AS ]  
BEGIN  
        function_body  
        RETURN scalar_expression  
END
```

Scalar Function – Tạo Hàm

- Ví dụ 1 : Hàm trả về tổng 2 số 4 và 6

Create function tong2so()

Returns int

as

Begin

 Declare @so1 int, @so2 int

 Set @so1 = 4

 set @so2 =6

 Return @so1+@so2

end

--thuc hien

print 'Tong = ' +convert(char(10),dbo.tong2so())

print 'Tong = ' +convert(char(10),tong2so())

select **dbo**.tong2so() as Tong

Scalar Function – Sửa và Xóa Hàm

Alter function tong2so()

Returns int

With Encryption

as

Begin

 Declare @so1 int, @so2 int

 Set @so1 = 4

 set @so2 =6

 Return @so1+@so2

End

--Xem lệnh

 sp_helptext tong2so

--thuc hien

 print 'Tong = ' +convert(char(10),dbo.tong2so())

 select dbo.tong2so() as Tong

--Xóa hàm Drop function Tong2so

Scalar Function

Ví dụ 2 : Hàm trả về tổng tiền của khách hàng có mã là TOMSP

Create function Tongtien()

Returns money

AS

Begin

 Declare @tong money

 Select @tong = sum(unitprice*Quantity) from orders o,
 [Order Details] d

 where o.orderid = d.orderid and customerid = 'TOMSP'

 Return @tong

End

print 'Tong = ' +convert(char(10),dbo.tongtien())

select dbo.tongtien() as [Tong Tien Cua Khach Hang TOMPS]

Scalar Function

- Scalar Function (Có tham số)
 - Là hàm nhận các giá trị từ bên ngoài truyền vào.
 - Cú pháp:

```
CREATE FUNCTION [owner_name.]function_name
([{@parameter_name [AS] data_type [=default]} [ ,...n ]])
RETURNS scalar_return_data_type
[WITH { ENCRYPTION | SCHEMABINDING } ]
[ AS ]
    BEGIN
        function_body
        RETURN scalar_expression
    END
```

Scalar Function

Ví dụ 3 : Hàm trả về tổng của hai số bất kỳ

```
Create function tong(@so1 int, @so2 int)
```

```
Returns int
```

```
as
```

```
Begin
```

```
    Return @so1+@so2
```

```
end
```

```
-- Thuc hien ham
```

```
Declare @a int, @b int
```

```
Set @a = 4
```

```
Set @b =6
```

```
Print 'Tong cua '+convert(char(5),@a) +' '+
```

```
convert(char(5),@b)+'='+convert(char(5),dbo.tong(@a,@b))
```

```
Select dbo.tong(@a,@b) as tong
```

Scalar Function

VD 4 : Hàm trả về tổng tiền của khách hàng nào đó

Create function TongtienTS(@makh nchar(5))

Returns money

AS

Begin

 Declare @tong money

 Select @tong = sum(unitprice*Quantity) from orders o, [Order Details] d
 where o.orderid = d.orderid and customerid = @makh

 Return @tong

End

declare @ma nchar(5)

Set @ma = 'TOMSP'

print 'Tong = ' +convert(char(10),**dbo.tongtiens(@ma)**)

select **dbo.tongtiens(@ma)** as Tong

Scalar Function

Bài tập áp dụng : Hàm trả về thứ bằng tiếng Việt

Create function thu(@ngay datetime)

Returns varChar(10)

As

```
Begin    Declare @t varchar(10), @d tinyint
         Set @d = datepart(dw,@ngay)
         Set @t = case
             When @d = 1 then 'Chu Nhat'
             When @d = 2 then 'Hai'
             When @d = 3 then 'Ba'
             When @d = 4 then 'Tu'
             When @d = 5 then 'Nam'
             When @d = 6 then 'Sau'
             When @d = 7 then 'Bay'
         end
         Return @t
end
```


Scalar Function

Example 6 : Hàm trả về thứ bằng tiếng việt

```
declare @ngaysinh datetime
Set @ngaysinh = getdate()      --hay '04/12/1982'--
Print 'Ban sinh vao Thu '+dbo.thu(@ngaysinh) +
    ' Ngay '+ convert(char(3),day(@ngaysinh)) + ' thang ' + Convert(char(3),
    month(@ngaysinh))+ ' nam ' +convert(char(5),year(@ngaysinh))

--thuc hien voi cau lenh Select
Select  employeeid,  LastName  + '      '+FirstName  as  Hoten,  thu  =
    dbo.thu(birthdate) from Employees
Select  employeeid,  LastName  + '      '+FirstName  as  Hoten,  [Thu Ngay Thang Nam
    Sinh]          ='Thu      '+dbo.thu(birthdate)          + '      Ngay      '+
    convert(char(2),day(birthdate))  + '      thang      '  +  Convert(char(2),
    month(birthdate))+ ' nam ' +convert(char(4),year(birthdate)) from Employees
```

Scalar Function

Bài tập 2 : Hàm trả về Tổng tiền của các sản phẩm

Create function TotalAmount

(@Unitprice money, @quantity Smallint,@Discount real)

Returns Money

As

Begin

Return (@Unitprice * @Quantity)*(1-@discount)

End

--Su dung

Select Productid, Total = dbo.TotalAmount(Unitprice,Quantity,Discount)

From [Order Details]

Where Orderid =10250

Scalar Function

Bài tập 3 :

Viết hàm trả về chiết khấu của sản phẩm dựa vào số lượng lập hoá đơn và theo quy định sau:

- Nếu số lượng ≤ 5 thì chiết khu là 0.05
- Nếu số lượng từ 6 đến 10 thì chiết khấu 0.07
- Nếu số lượng từ 11 đến 20 thì chiết khấu là 0.09
- ngược lại thì 0.1

Scalar Function

Bài tập 3 :

Viết hàm trả về chiết khấu của sản phẩm dựa vào số lượng lập hoá đơn và theo quy định sau:

- Nếu số lượng ≤ 5 thì chiết khu là 0.05
- Nếu số lượng từ 6 đến 10 thì chiết khấu 0.07
- Nếu số lượng từ 11 đến 20 thì chiết khấu là 0.09
- ngược lại thì 0.1

The table-valued UDFs

- The table-valued UDFs: **được chia thành hai loại là inline và multistatement table-valued.**
- *Inline table-valued UDF:*
 - *Được xem như là một View có tham số. Thực thi một câu lệnh Select như trong một view nhưng có thể bao gồm các tham số giống thủ tục*
 - **Cú pháp:**

```
CREATE FUNCTION [owner_name.]function_name  
([{@parameter_name [AS] data_type [=default]} [ ,...n ]])  
RETURNS TABLE  
[WITH { ENCRYPTION | SCHEMABINDING }]  
[AS]  
RETURN [(] select-stmt [)]
```

The table-valued UDFs

Ví dụ 1: Cho biết tổng số hóa đơn của khách hàng bất kỳ.

```
CREATE FUNCTION CountOrderCust (@cust varchar(5))  
RETURNS TABLE  
AS  
RETURN (Select CustomerID, count(orderid)as countOrder  
        From orders  
        Where customerID like @cust  
        Group by customerID )
```

Thi hành (không cần tên đầy đủ)

```
Select * from CountOrderCust('A%' ) --Loi  
declare @ma nvarchar(5)  
Set @ma='A%'  
select * from CountOrderCust(@ma)
```

The table-valued UDFs

Ví dụ 2 : trả về tổng số lượng của từng sản phẩm theo loại hàng nào đó.

```
CREATE FUNCTION SalesByCategory(@Categoryid Int)
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN
```

```
(SELECT c.CategoryName, P. ProductName,
```

```
        SUM(Quantity) AS TotalQty
```

```
FROM Categories c
```

```
INNER JOIN Products p ON c.CategoryID= p. CategoryID
```

```
INNER JOIN [Order Details] od ON p.ProductID = od.ProductID
```

```
WHERE c.CategoryID= @Categoryid
```

```
GROUP BY c. CategoryName,p.ProductName)
```

- Thực thi

```
SELECT * FROM SalesByCategory (1)
```

The table-valued UDFs

- **Multistatement Table-valued UDF:** là dạng phức tạp nhất. Loại hàm này xây dựng tập kết quả từ một hay nhiều câu lệnh Select

- Cú pháp:

```
CREATE FUNCTION [owner_name.]function_name  
([{@parameter_name [AS] data_type [=default]} [ ,...n ]])  
RETURNS @return_variable  
TABLE ({column_definition | table_constraint} [ ,...n ])  
[WITH { ENCRYPTION | SCHEMABINDING } ]  
[AS]  
BEGIN  
    function_body  
RETURN  
END
```


The table-valued UDFs

- Ví dụ 1

```
CREATE FUNCTION CountOrderCust()  
RETURNS @fn_CountOrderCust TABLE  
(OrderIdent tinyint Not null, Cust varchar(5) )  
AS  
Begin  
    Insert @fn_CountOrderCust  
    Select Count(orderid),CustomerId From Orders Group by customerid  
    Return  
end
```

- --Thi hành

```
Select * from CountOrderCu()
```

The table-valued UDFs

- Ví dụ 2

```
CREATE FUNCTION Contacts(@suppliers bit=0)
RETURNS @Contacts TABLE (ContactName nvarchar(30), Phone nvarchar(24), ContactType nvarchar(15))
AS BEGIN
    INSERT @Contacts
    SELECT ContactName, Phone, 'Customer' FROM Customers
    INSERT @Contacts
    SELECT FirstName + ' ' + LastName, HomePhone, 'Employee'
    FROM Employees
    IF @Suppliers=1
    INSERT @Contacts
    SELECT ContactName, Phone, 'Supplier'
    FROM Suppliers
RETURN
END
▪ Thực thi
SELECT * FROM CONTACTS(1) ORDER BY ContactName
```

Sử dụng UDFs

- **A scalar UDF:** khi gọi luôn luôn theo cú pháp: *owner.functionname*.

Ví dụ:

```
SELECT ProductID, Total=dbo.TotalAmount(UnitPrice, Quantity, Discount)
FROM [Order details]
WHERE OrderID=10250
```

- **A scalar UDF có thể được sử dụng trong biểu thức, trong câu lệnh SELECT hay lệnh CREATE TABLE**

```
CREATE TABLE [Order Details] (
    OrderID int NOT NULL , ProductID int NOT NULL ,
    UnitPrice money NOT NULL DEFAULT (0),
    Quantity smallint NOT NULL DEFAULT (1),
    Discount real NOT NULL DEFAULT (0),
    Total AS dbo.TotalAmount(UnitPrice, Quantity, Discount))
```

Using UDFs

- **A table-valued UDF:** Có thể được gọi theo cú pháp *owner.functionname* hay *functionname*

```
SELECT * FROM Contacts(1) ORDER BY ContactName
```

- Nếu table-valued function không có tham số, bạn phải sử dụng dấu()
- Nếu tham số có giá trị mặc định, bạn phải truyền giá trị vào mặc dù bạn có sử dụng từ khóa DEFAULT

```
SELECT * FROM Contacts() ORDER BY ContactName
```

Using UDFs

- Viết hàm trả về danh sách các hoá đơn đã lập của một khách hàng nào đó trong một tháng năm nào đó. Thông tin gồm: Makh, TenKh, Diachi, mahd, ngaylapHD, Noichuyen, LoaiHD. Trong đó, LoaiHD được hiển thị rõ là Nhập hoặc Xuất.

Chuyển thủ tục thành hàm người dùng

- Hàm người dùng có giá trị trả về dạng bảng có thể thay thế cho bất kỳ thủ tục nào trả về 1 bộ kết quả. Bảng được trả về bởi hàm có thể được dùng trong mệnh đề FROM của lệnh SQL, còn bộ kết quả được trả về bởi thủ tục thì không thể sử dụng được.
- Nếu muốn gọi một thủ tục trực tiếp từ 1 query thì cần chuyển thủ tục thành hàm người dùng.

Chuyển thủ tục thành hàm người dùng

- Nếu 1 thủ tục thỏa mãn các tiêu chuẩn sau, thì dễ dàng chuyển nó thành hàm dạng bảng:
 - Nếu 1 thủ tục có chứa 1 lệnh SELECT đơn để biểu diễn 1 view có tham số.
 - Thủ tục trả về 1 bộ kết quả (result set)
 - Mục đích chính của thủ tục là nhằm tạo 1 bộ kết quả trung gian được lưu vào 1 bảng tạm thời, sau đó sẽ được truy vấn bởi lệnh SELECT. Khi đó lệnh INSERT...EXEC có thể được viết lại thành hàm.

Chuyển thủ tục thành hàm người dùng

- Xét đoạn lệnh sau:

```
INSERT #temp EXEC sp_getresults  
SELECT ...  
FROM #temp, t1  
WHERE ...
```

- Thủ tục sp_getresults có thể được viết thành hàm dạng bảng fn_results(), khi đó đoạn lệnh trên có thể được viết lại thành:

Bài tập

1. Viết hàm tên SubTotalOfEmp (dạng scalar function) trả về tổng doanh thu của một nhân viên trong một tháng tùy ý trong một năm tùy ý, với tham số vào @EmpID, @MonthOrder, @YearOrder (Thông tin lấy từ bảng [Sales].[SalesOrderHeader])
2. Viết hàm tên là InventoryProd (dạng scalar function) với tham số vào là @ProductID và @locationID trả về số lượng tồn kho của sản phẩm trong khu vực tương ứng với giá trị của tham số (Dữ liệu lấy từ bảng [Production].[ProductInventory])

Bài tập

1. Viết hàm sumofOrder với hai tham số @thang và @nam trả về danh sách các hóa đơn (SalesOrderID) lập trong tháng và năm được truyền vào từ 2 tham số @thang và @nam, có tổng tiền >70000, thông tin gồm SalesOrderID, Orderdate, SubTotal, trong đó $SubTotal = \text{sum}(\text{OrderQty} * \text{UnitPrice})$.
2. Viết hàm tên SumofProduct với tham số đầu vào là @MaNCC (VendorID), hàm dùng để tính tổng số lượng (sumOfQty) và tổng trị giá (SumofSubtotal) của các sản phẩm do nhà cung cấp @MaNCC cung cấp, thông tin gồm ProductID, SumofProduct, SumofSubtotal (sử dụng các bảng *[Purchasing].[Vendor]* *[Purchasing].[PurchaseOrderHeader]* và *[Purchasing].[PurchaseOrderDetail]*)

Bài tập

1. Viết hàm sumofOrder với hai tham số @thang và @nam trả về danh sách các hóa đơn (SalesOrderID) lập trong tháng và năm được truyền vào từ 2 tham số @thang và @nam, có tổng tiền >70000, thông tin gồm SalesOrderID, Orderdate, SubTotal, trong đó $SubTotal = \text{sum}(\text{OrderQty} * \text{UnitPrice})$.
2. Viết hàm tên SumofProduct với tham số đầu vào là @MaNCC (VendorID), hàm dùng để tính tổng số lượng (sumOfQty) và tổng trị giá (SumofSubtotal) của các sản phẩm do nhà cung cấp @MaNCC cung cấp, thông tin gồm ProductID, SumofProduct, SumofSubtotal (sử dụng các bảng *[Purchasing].[Vendor]* *[Purchasing].[PurchaseOrderHeader]* và *[Purchasing].[PurchaseOrderDetail]*)

Tài liệu tham khảo

- Raghu Ramarkrishnan, Johannes Gehrke. *Database Management System*. 3rd Edition, Mc Graw Hill, 2003.
- Tutorials Point, Database Management System, www.tutorialspoint.com, 2015.
- Hector Garcia-Molina, Jeffrey D. Ullman and Jennifer Widom. *Database Systems, the Complete Book*, 2nd Edition, Prentice Hall, 2009
- Ramez Elmasri, Shamkant B. Navathe. *Fundamentals of Database Systems*. 7th Edition, Pearson, 2015
- Silberschatz, Korth and Sudarshan, *Database System Concepts* (6th Edition), McGraw-Hill, 2010.

Hết!
Cảm ơn Anh/ Chị đã theo dõi!

GVHD: TS. Phan Thị Thể
FIT – HCM-UTE
Email: thept@hcmute.edu.vn